

Member Callout

Session Report: Requirements, Design, Build, and Verification

Avialdo Solutions Technical Exercise · CrewLink / Local 27 Callout System

Field	Value
Repository	github.com/mhassanahmed8537/crewlink
Stack	Django, DRF, Postgres, Celery, Redis, Next.js, nginx
Scope covered	Part 0 (requirements), Part A (design), Part B (build), devops bonus
Final commit	2fab02a (merged, main)
Status	All automated tests passing; verified end to end against a clean rebuild

This document is a full record of one working session: reviewing a messy stakeholder ask, turning it into requirements and a design, building a running slice of that design, and verifying it end to end against a completely clean environment before pushing it to a shared repository.

Executive Summary

CrewLink is a fictional platform for labour unions. A business manager, Denise Okafor of Local 27, sent a short, informal message describing three real problems: members missing emergency callouts, no way to know who is actually coming to a callout before leaving the hall, and a possible access control leak letting members outside her local see apprentices' contact information. This session took that message through the full lifecycle the exercise asks for: restating it as requirements, designing a system against two hard rules, building a running slice of that system, and verifying the whole thing end to end against a genuinely clean environment, not just the environment it happened to be built in.

What this session produced

- A requirements section restating the ask, listing assumptions, flagging a real access control concern in Denise's message, and recording three questions that could not be asked.
- A design covering the data model, the send path from a button press to 22,400 phones, how both rules are enforced structurally rather than by convention, and a diagram of the running system.
- A working backend (Django, DRF, Postgres, Celery, Redis) and a working frontend (Next.js) implementing that design, including the AI drafting feature and the devops bonus.
- Six automated tests covering both rules, plus a full manual verification pass repeated twice against a completely wiped and rebuilt environment.
- One real bug found and fixed by testing in an actual browser rather than trusting API tests alone (a missing CORS header), and one real bug found and fixed before it could reach a reviewer (non-deterministic seed data that would have made the README's documented ids wrong on every fresh clone).
- A public GitHub repository with a clean, logically organized commit history, reconciled after an unrelated-history divergence with zero actual content conflicts.

1. The Assessment Brief

1.1 The ask, as given

The following is the stakeholder message the entire exercise starts from, reproduced as given, attributed to Denise Okafor, Business Manager, Local 27 (fictional):

"Hey, we keep missing people on the callouts. Half of Local 27 didn't know about Thursday's emergency meeting until it was too late and it's caused real problems with the contractor situation. Members are saying they never got anything, or they got it but by the time they saw it the meeting was already over. Also I need to know before I leave the hall on the day of a callout who's actually coming so I'm not standing in an empty room. Oh and can members outside our local see the wireman apprentices' contact info too? someone mentioned that during onboarding and I want to make sure that's not happening. We need this sorted before the next round of contract talks, whenever that ends up being scheduled."

1.2 The numbers the system is designed for

Metric	Value
Union locals on one platform	43
Members total	187,000
Largest local (one callout goes to all of them)	22,400
Push notifications that silently fail	roughly 5 to 10 percent
How long a member can be offline	hours, sometimes days

1.3 The two rules that shape everything

Rule 1

Nobody sees or touches data outside their own access. One local's information must never reach another local's members or leadership, and a member must never be able to act with a leader's permissions.

Rule 2

A retry or a restart must not deliver twice to the same member. Leadership will retry a send that looks stuck, and a crashed process will restart mid send. 22,400 people must not get it twice either way.

1.4 What the brief asked for

Part	Ask	Weight
Part 0	Turn the message into a short requirements section: restate it, list assumptions, flag a concern, write three unanswerable questions.	~45 min
Part A	Design: data model, the send path, how both rules are enforced, one diagram.	~1.25 hr
Part B	Build a thin slice: backend with both rules genuinely enforced and one AI feature, a single leadership screen, seed data, a README with test accounts.	~3.5 hr

The brief is explicit that Part 0 and Part A carry the most weight, and that Part B's build time should not crowd out design time.

2. Reviewing the AI Design Sessions Against the Brief

Before writing the final requirements and design, five prior AI chat sessions (exported as PDFs and included at the repository root, per the brief's section 5) were reviewed against the brief's exact instructions, not just for correctness, but for whether they respected the boundary between Part 0 (restate and flag, a few sentences each) and Part A (design decisions and rejected alternatives).

2.1 Session 1, Part 0 requirements

Correctly identified all four required elements and, notably, landed on exactly the concern the brief is fishing for: the apprentice contact info question is a live Rule 1 violation, not a minor cleanup item. The session's weakness was scope discipline: several answers pre committed to specific technical mechanisms (a store and forward queue, a named RBAC scheme, a 30 second delivery budget) inside what should have been a short assumptions section. This was corrected in the final DESIGN.md by trimming Part 0 to plain restatement and one line assumptions, and relocating the technical reasoning to Part A, where the brief actually scores it.

2.2 Session 2, data model

Proposed RSVP as a fixed three state enum rather than a boolean or free text, and a role field on members rather than a separate leadership table, both adopted as is. Also proposed the unique constraint on (announcement_id, member_id) with an insert that skips conflicts, which became the core mechanism Rule 2 rests on.

2.3 Session 3, the send path

Directly answered the brief's specific question about four people watching a live status screen, proposing a short lived cache recomputed by the worker rather than a live query per poll or a websocket. Adopted as is, including the explicit reasoning for rejecting websockets as more infrastructure than this exercise calls for.

2.4 Session 4, the two rules

Produced the base viewset and permission class pattern for Rule 1, plus an actual pytest suite proving the boundary, both adopted. For Rule 2, the session itself surfaced a real gap: the unique constraint protects against a restarted worker but not against a client retrying the original request, since a retry would otherwise create a second announcement with a new id, invisible to the recipient level constraint. The session flagged this but did not resolve it. The final design closes it by adding a client supplied request id, unique per local, so a repeated request is recognized before a second announcement is ever created.

2.5 Session 5, the diagram

Asked explicitly to diagram only what the earlier sessions had actually described, not invented extras, and delivered exactly that: client, load balancer, two app instances, Postgres, Redis, the worker, and the AI provider call, each connection labeled synchronous or asynchronous.

3. Part 0, Requirements (final)

What I am building

A way for local leadership to send an emergency notice to some or all of their members, know how many actually saw and acknowledged it before a callout, and be certain that a local's member data, and a member's ability to act, never crosses into another local's territory. The system has to survive a member being off the network for hours or days, and a retried or restarted send cannot reach the same person twice.

Assumptions

Scope

The design targets all 43 locals and 187,000 members, the numbers given in the brief. Local 27 is the only tenant that gets used and seeded for the build slice.

What "immediately" means

Best effort delivery within seconds for devices that are online, with the notice queued and flushed the moment an offline device reconnects. This is not a guarantee, since 5 to 10 percent of push notifications silently fail and some members are unreachable for days.

Whether RSVP is required

Optional, not enforced. Leadership cannot compel a mandatory response from someone who is off grid. Delivered, read, and acknowledged state is tracked instead, so Denise gets an honest headcount rather than a false one.

What counts as "done"

A callout is considered resolved at the meeting's scheduled time, not at 100 percent delivery, since full delivery may never happen. The counts stay visible as a snapshot after that point.

Concern worth flagging

Denise's question about apprentice contact info is not a minor onboarding cleanup item. If it is true that members outside her local can see it, then one local's member data is already reachable from outside that local, which is exactly what Rule 1 exists to prevent. This is treated as a real access control defect to design against from the start, not a hypothetical.

Questions that could not be asked, and what was assumed instead

- **Is RSVP confirmation required before a callout, or just useful to have?** Assumed optional and informational, since a mandatory response cannot survive members being legitimately offline.
- **Does "everyone" in a callout ever mean sending across multiple locals at once?** Assumed no. A callout is scoped to one local by default, optionally narrowed by classification. Cross local broadcast is out of scope for this exercise.
- **What delivery latency actually counts as acceptable, given that some members are offline for days?** Assumed that online devices are notified within a best effort window of seconds, and that a notice counts as delivered once the device has actually received it, however long that takes.

4. Part A, Design (final)

4.1 Data model

The starting schema was missing two things on purpose: a way to tell a leadership account from a member account, and a place to record what each recipient actually did with a message. Both additions are small changes to the given tables, not a rewrite.

```
locals
  id, name

members
  id, local_id, full_name, email, classification, status, role

announcements
  id, local_id, title, body, needs_ack, sent_at, created_at, request_id

announcement_recipients (new)
  id, announcement_id, member_id, delivery_status, sent_at, read_at,
  acknowledged_at, rsvp, rsvp_updated_at
```

role on members is member or leadership. A leader is still tied to one local and shares every field a member has, so a flag on the existing table avoids a second table and an extra join on every permission check.

request_id on announcements is a value the client generates once per create attempt, unique per local. It exists so a retried create request is recognized as the same request instead of producing a second announcement, which matters for Rule 2.

announcement_recipients is the table that answers Denise's actual question. `delivery_status` and the three timestamp columns record whether a member has done nothing, been sent the message, read it, or acknowledged it. `rsvp` is a fixed three value field (coming, cant, no_response) rather than free text or a boolean, because a boolean cannot separate an explicit decline from silence. The pair of `announcement_id` and `member_id` is unique, which is what Rule 2 rests on.

4.2 The send path

When leadership presses Send, the request that hits the server does three things and nothing else: check the caller is authenticated leadership for that local, validate the title and body, and insert one row into announcements with `sent_at` set. It then hands off a background job with the new announcement's id and returns. That whole path targets well under a second, regardless of whether the audience is twenty members or 22,400, because the request never touches a recipient row.

Everything involving the 22,400 phones happens after the response, in a worker. The worker looks up the active members for that local, optionally narrowed by classification, and inserts one recipient row per member in batches of about a thousand, using a conflict safe insert. As each batch lands, it hands the member ids in that batch to a dispatch step that marks each row sent or failed. None of this blocks leadership's screen, which already has its response.

A member offline for hours or days is not treated as a delivery failure; a row can sit at sent for a long time before it becomes read. failed and undeliverable are reserved for the roughly 5 to 10 percent of sends the brief calls out as silently dropped by the provider itself. Those rows are swept and retried once, a few minutes later, before being left as a final, visible failure.

For the status screen, several people watching sent, read, and acknowledged counts refresh every few seconds is a real load risk if each refresh runs a fresh aggregate query. The counts are instead recomputed once, by the worker, after each batch, and cached for a few seconds; the status endpoint reads that cached value first and only falls back to a live query if the cache is empty.

4.3 The two rules, by design

Rule 1 is enforced at one place: a base viewset that every endpoint touching local scoped data must inherit from. It filters every query by the caller's own `local_id` before anything else runs, and a permission class layered on top requires role equal to leadership for actions only leadership should reach. A new endpoint is only unsafe if whoever writes it skips those base classes and talks to the model directly, a visible choice in a diff, not a forgotten line inside a view.

Rule 2 rests on two constraints, not application logic. The unique pair of `announcement_id` and `member_id` on `announcement_recipients`, combined with an insert that does nothing on conflict, means a worker that crashes partway through can simply restart from the beginning; already written rows are silently skipped. That covers a restarted worker, but not a retried request from leadership, since a second identical POST would otherwise create a second announcement with its own id. The client supplied `request_id` closes that gap: a repeated `request_id` for the same local is treated as the original request, not a new announcement.

Detecting a broken rule in production, without reading code: for Rule 1, a periodic join of `announcement_recipients` back to its announcement's `local_id` and its member's `local_id`, checking for any row where the two differ, catches a leak after the fact. For Rule 2, a query grouping by `announcement_id` and `member_id` and checking for a count above one should always return nothing while the constraint is in place, so the practical production signal is a unique violation appearing in the database logs, not a scheduled query expected to stay empty.

4.4 Diagram

```
graph TD
    Client["Next.js client"]
    LB["Load balancer (nginx)"]
    App1["Django and DRF, instance 1"]
    App2["Django and DRF, instance 2"]
    DB["Postgres"]
    Queue["Redis"]
    Worker["Celery worker"]
    AI["External AI provider"]

    Client -->|sync| LB
    LB -->|sync| App1
    LB -->|sync| App2
    App1 -->|sync| DB
    App2 -->|sync| DB
    App1 -->|sync, enqueue| Queue
    App2 -->|sync, enqueue| Queue
    Worker -->|async, dequeue| Queue
    Worker -->|sync| DB
    App1 -->|sync, draft request| AI
    Worker -->|sync, draft request| AI
```

Two Django instances sit behind nginx so the devops bonus, hitting either instance at random, has something to hit. The worker is the only path that ever writes 22,400 recipient rows, and it talks to the same Postgres the two Django instances use, so the unique constraint holds no matter which instance took the original request. The diagram shows one worker as a starting point, not a ceiling: Rule 2 is enforced by a database constraint, not by anything a worker holds in memory, so running several workers against the same queue needs no

design change.

5. Part B, The Build

A running slice of the design above, not the whole thing: the spine, front end included, matching the brief's own scope relief (unstyled UI is fine, no member facing screen required, real push credentials not required, runs locally with one command).

5.1 Backend

- **Stack:** Django 5, Django REST Framework, PostgreSQL, Celery, Redis, token authentication.
- **Data model:** Local, Member (the custom auth user, carrying local, role, classification, status), Announcement (with request_id for retry safety), AnnouncementRecipient (with the unique announcement/member constraint Rule 2 rests on).
- **Rule 1** enforced by LocalScopedQuerysetMixin (filters every queryset by the caller's local_id) and IsLocalLeadership (a permission class requiring role == leadership), combined in a shared LeadershipScopedViewSet every relevant endpoint inherits from.
- **Rule 2** enforced by the unique constraint on (announcement, member) plus batched bulk_create(..., ignore_conflicts=True) in the Celery task, and the request_id idempotency check on the create endpoint (an Idempotency-Key header, or a body field, resolved to the existing announcement rather than a new one).
- **Async send path:** the create endpoint validates and inserts the announcement row, enqueues a Celery task, and returns; the task fans out recipient rows in batches, simulates dispatch at a configurable failure rate matching the brief's 5 to 10 percent number, and schedules one retry sweep for anything that failed.
- **Status caching:** counts are recomputed by the worker after every batch and cached in Redis with a short TTL; the status endpoint reads the cache first and only queries Postgres directly if the cache has expired.
- **The AI feature:** a stateless draft endpoint that turns a messy note into a title, body, and a push preview capped at 120 characters, calling Anthropic's API with an explicit timeout. On any failure, timeout, missing key, malformed response, it returns {"available": false} rather than an error or a fabricated fallback, and never sends anything; sending only happens through the separate create endpoint that leadership calls after reviewing and, if needed, editing the draft.
- **Seed data:** a re-runnable management command creating two locals (about 2,000 members and 200 members, across four classifications), one leadership and one member login per local, and one already sent announcement with full recipient rows in the larger local. Ids for everything the README quotes by value are derived deterministically with uuid5 from a fixed namespace, so a completely fresh clone produces the same ids, not random ones.

5.2 Frontend

One Next.js screen, deliberately unstyled beyond basic readability. Leadership signs in, can paste a messy note and generate an AI draft (or write a title and body directly), optionally narrow the audience by classification, and hit Send. A status panel then appears and polls the announcement's counts every three seconds, showing total, sent, read, acknowledged, and failed, with no page reload.

The screen does not include a local picker. The brief's wording describes leadership picking a local, but Rule 1 ties a leadership login to exactly one local, so there is nothing to pick; the screen shows the caller's own local as

a fact. This is documented as a deliberate deviation in DESIGN.md, not an oversight.

5.3 Infrastructure

A single docker-compose.yml runs the whole stack: Postgres and Redis with health checks, a one-shot migrate service that runs migrations and seeds data before anything else starts, two backend instances (web1, web2), a Celery worker, an nginx load balancer round-robinning between the two backend instances, and the frontend. The devops bonus is fully built, not just designed: a retried send proven not to double deliver while genuinely hitting either backend instance at random through the load balancer.

6. Verification, End to End

The system was verified twice from a completely clean state (volumes wiped, images rebuilt from scratch), the same experience a reviewer's first ever docker compose up would produce, not just re-checked in an environment that had accumulated state from earlier runs.

6.1 Automated tests

Six pytest tests, run against a real Postgres, not mocked:

- A leadership account's list and detail views exclude another local's announcements entirely.
- A leadership account cannot retrieve another local's announcement by id (404).
- A member account cannot create an announcement (403).
- A member account cannot read an announcement outside their own local (404).
- A retried create request, same Idempotency-Key, returns the original announcement and does not double the recipient rows.
- A restarted worker, `dispatch_announcement` called twice for the same announcement, does not double insert recipient rows.

Result: **6 passed**, confirmed on two separate clean rebuilds.

6.2 Manual verification, by curl and by browser

Check	Method	Result
Login, all four seeded accounts	curl	All succeed, correct role and local returned
Rule 1, cross-local leadership read	curl	404
Rule 1, cross-local member read	curl	404
Rule 1, member attempts to create an announcement	curl	403
Rule 2, same Idempotency-Key sent twice through the load balancer	curl (round-robin across both instances)	Same announcement id both times; recipient count not doubled
Member acknowledge flow, using the exact id from the README	curl	Recipient row updated; documented id matched a fresh login exactly
AI draft, no provider configured	curl and a real browser click	Graceful "available: false", never a crash, never a silent send
Full leadership screen	Real browser clicks: sign in, paste note, generate draft (fallback shown), type title/body, hit Send	Status panel appeared and updated live counts every 3 seconds with no reload

6.3 Bugs found during verification, and how

CORS was silently blocking the send button

The API worked perfectly by curl, but clicking Send in an actual browser failed with a network level error. The preflight OPTIONS request returned 200, but the browser still blocked the real POST, because django-cors-headers' default allowed header list does not include a custom Idempotency-Key header. This was only caught by testing in a real browser and reading the network request log; an API-only test suite would never have surfaced it. Fixed by explicitly adding the header to CORS_ALLOW_HEADERS.

Seed data used random ids

The seed script originally let locals, the four named logins, and the pre-sent announcement get random UUIDs. That meant the ids hardcoded into the README's required Test Accounts block would only ever be correct for the one database they were copied from; a reviewer's own fresh docker compose up would generate different ids and every curl example in the README would 404. Fixed by deriving those specific ids deterministically with uuid5 against a fixed namespace, then proven by wiping the database entirely and confirming the new run produced the exact same ids already written into the README.

Dark on dark text in the frontend

The page never set an explicit background color, so it inherited a dark host theme while text stayed near black, rendering the screen almost unreadable. Caught with an actual screenshot, not just a text dump of the page, and fixed with an explicit light background and color-scheme on the page body.

7. Git History and Deployment

The repository was initialized locally, committed in four logical stages mirroring the actual order of work, and pushed to a pre-existing GitHub repository provided by the user.

```
a4e94a2  Add Part 0 requirements and Part A design
30f6c1c  Build Part B backend: Django, DRF, Postgres, Celery
bde6169  Add the Next.js leadership screen
8cb340c  Wire up docker compose, the nginx load balancer, and the README
```

7.1 Reconciling a divergent remote

On a later push, origin/main had independently accumulated four commits carrying the same messages as the local history but with no shared ancestor. Before merging anything, the two trees were diffed directly, confirming zero actual content differences, this was a metadata level divergence, not a real conflict. The two histories were merged with `--allow-unrelated-histories`, which resolved cleanly with no conflicts to fix by hand, and the full test and verification pass was repeated in full against the merged result before pushing again.

```
* 2fab02a  Merge origin/main: reconcile independently produced
|\      but identical history
| * 48dafca  Wire up docker compose, the nginx load balancer, and the README
| * 5fd7303  Add the Next.js leadership screen
| * 032f63e  Build Part B backend: Django, DRF, Postgres, Celery
| * d66e6ad  Add Part 0 requirements and Part A design
* 8cb340c  Wire up docker compose, the nginx load balancer, and the README
* bde6169  Add the Next.js leadership screen
* 30f6c1c  Build Part B backend: Django, DRF, Postgres, Celery
* a4e94a2  Add Part 0 requirements and Part A design
```

Final state: github.com/mhassanahmed8537/crewlink, branch main at 2fab02a, public, working tree clean, no secrets committed (only .env.example templates, never a populated .env).

8. Final Compliance Check

8.1 Against the brief

Requirement	Status
Part 0: restate, assumptions, flagged concern, three questions	Done, in DESIGN.md
Part A: data model, send path, both rules, one diagram, under 1,500 words	Done, roughly 1,415 words across the four prompts
Part B backend: create and send, per-recipient records, member read/acknowledge, both rules genuinely enforced, one AI feature	Done and verified live
Part B frontend: one leadership screen, title/body or messy note, pick audience, send, live counts	Done, verified by real browser clicks
Stack: Django, DRF, Postgres	Matches exactly
Seed data: two locals, ~2,000 and ~200 members, 3 to 4 classifications, one login pair per local, one already-sent announcement with recipient rows	Done, deterministic ids
README: run instructions, logins, curl examples, cross-local proof, Rule 2 proof, TEST ACCOUNTS block	Done, exact heading present
Devops bonus: two instances behind a load balancer, proven no double send	Built, not just designed; proven by an automated test and a live curl sequence
AI conversations exported to repo root	Done, five sessions
What was cut, noted at the bottom of DESIGN.md	Done
Single GitHub repo, normal commit history	Done: github.com/mhassanahmed8537/crewlink

8.2 Against Denise's message, point by point

Denise's problem	Resolution
Members miss callouts, half of Local 27 didn't know in time	Full audience fan out with per-recipient delivery tracking distinguishing never delivered from delivered late
Members say they never got anything, or got it too late	delivery_status and read_at now make these two distinct, visible states instead of one undifferentiated complaint
Needs to know who's coming before leaving the hall	Modeled fully (RSVP field); the running screen shows live acknowledged count as the headcount signal, since RSVP itself is explicitly design-only for this build per the brief
Can members outside the local see apprentice contact info	Solved and proven: cross-local access returns 404 for both leadership and members, confirmed by automated tests and live curl calls
Needs this before the next round of contract talks	A deadline, not a technical requirement, correctly left alone

9. What Was Cut, and What's Next

- **A second delivery channel.** A row still failed after one retry stays failed; no SMS or other fallback channel was built. One retry against the same channel was judged the honest line between a real fix and scope creep for this exercise.
- **RSVP enforcement.** Modeled in the data and seeded with realistic values, but no endpoint lets a member set it, per the brief's own instruction that RSVP is design only.
- **A member-facing screen.** Optional per the brief; the member side is endpoints only, exercised through curl and pytest.
- **A locals picker.** Intentionally omitted since Rule 1 ties a leadership login to exactly one local; there is nothing to pick.
- **Real push delivery.** Simulated at the brief's own stated failure rate and logged, per the brief's scope relief; no FCM or APNs credentials used.

Next steps, if this continued

- A second delivery channel for whatever is still undeliverable after the one retry.
- The classification list read from the database instead of matched by hand between the seed script and the frontend dropdown.
- Real push provider credentials behind the same dispatch interface that already exists, since that interface was written to make the swap a small change, not a rewrite.